

The AI/ML Engineer Interview Playbook

PART 1

First-Round Questions and Battle-Tested Answers
Projects | GenAI Fundamentals | Voice AI Agents

Curated from real interview experience

@aiml_rohandevaki

How to Use This Guide

In 2026, AI engineering interviews are 60%+ GenAI focused. The old mix of CNNs, tree-based models and gradient descent now covers barely a quarter of technical rounds. What interviewers actually test: can you build, evaluate, and debug LLM-powered systems under real cost and latency constraints?

Three rules that apply to every answer in this guide:

1. Lead with your own project or work experience. It builds instant trust and credibility. The interviewer knows you have actually shipped something, not just read about it.

2. Use the PITCH framework when explaining any project. Do not just explain your project. PITCH it.

P: Problem statement. One crisp sentence on the business problem you solved.

I: Individual role. What YOU personally owned, not what the team did.

T: Thought process. Options you considered, trade-offs, and why you chose your approach.

C: Collaboration. Which team members you worked with (backend, data, product) and how.

H: Hard numbers. Quantified impact. Example: "I implemented streaming and brought TTFT down to 0.5 seconds."

3. Use STAR for story questions (Situation, Task, Action, Result) and always put a number on the result. "Cut latency from 4s to 0.5s TTFT" beats "it got faster" every single time.

Bonus rule: add production context. Theoretical answers without cost, latency, evaluation, or failure-mode discussion are the number one reason candidates fail GenAI interviews.

Section 1: Project and Experience Questions (All Levels)

Q1. Explain your most recent AI/ML project.

Answer using PITCH, in this exact order. Interviewers steer the whole round based on this answer.

P, Problem statement: one crisp sentence on the business problem. **I, Individual role:** what you personally owned. **T, Thought process:** options considered, trade-offs, why you chose your path. **C, Collaboration:** which team members you worked with (backend, data, product) and how. **H, Hard numbers:** quantified value. Example: "I implemented streaming in the response pipeline and brought TTFT down to 0.5 seconds, making conversations feel real-time."

Pro tip: Most follow-up questions come from whatever YOU say here. Only mention components you can defend three levels deep.

Q2. Why did you choose LangGraph over LangChain?

LangGraph is a **stateful graph execution framework** that explicitly supports **cycles**: loops, retries, reflection, and conditional routing between nodes. That is exactly what agentic workflows need: an agent that calls a tool, evaluates the result, and decides to retry or move on.

LangChain, by contrast, is an orchestration library for composing mostly **linear pipelines**: loading, chunking, indexing, retrieval, and answer generation. It is great for straightforward RAG chains but has no first-class support for iterative cycles or shared state across steps.

In short: LangChain for linear pipelines, LangGraph when you need state, branching, and iteration.

Pro tip: Bonus point: mention that LlamaIndex is specialized for ingestion and indexing. It shows you know the full ecosystem.

Q3. How did you reduce latency? (STAR example)

Situation: While building a voice AI agent, every conversation turn had around 4 seconds of latency between the user finishing speaking and the agent responding.

Task: Bring it to sub-second so conversations feel natural.

Action: Profiled the full pipeline component by component, STT to LLM to TTS, to find the biggest contributor. Root cause: a single monolithic agent with a roughly 30k-token prompt handling every responsibility. I redesigned it as a multi-agent architecture with specialized agents, cutting the active prompt to around 10k tokens, and enabled streaming so TTS starts speaking on the first tokens.

Result: Turn latency dropped from 4s to 0.5s TTFT, and each specialized agent handled its flow more reliably than the overloaded single agent.

Pro tip: Always name the measurement first ("I profiled each stage"). It shows engineering discipline, not guesswork.

Q4. Which agentic architecture did you choose, and why?

Show breadth first, then justify your pick: single-agent, multi-agent (supervisor or coordinator), reactive (ReAct-style), hierarchical or tree-based, and router patterns.

Then explain that your decision came from evaluating the problem against trade-offs: **latency** (more agents means more LLM calls), **token and memory cost**, **reliability** (smaller focused prompts fail less), and **debuggability**. Example: "I moved to a supervisor plus specialized agents pattern because the single agent's prompt was overloaded. The split reduced both latency and error rate."

Q5. How did you improve RAG accuracy to 95%?

Context: A transportation-domain project. The goal was to determine whether a property owner is impacted by a flyover or road project.

Problem found: Vanilla vector retrieval failed on exact-match queries such as specific owner names, survey numbers, and property IDs. Semantic similarity is weak on rare keywords.

Solution: First, broke large satellite imagery into tiles and structured each property's information as a JSON payload attached to its vector point. Second, implemented **metadata and payload filtering** so exact fields (owner name, ID) filter candidates alongside semantic search, which is effectively hybrid retrieval.

Result: Accuracy improved to 95% on the evaluation set.

Pro tip: Be ready for the follow-up: "95% measured how?" Have your eval set size and metric (exact match, LLM-as-judge, or human labels) ready.

Q6. Explain a production issue and how you fixed it.

Interviewers probe four things, so prepare all four in advance: (1) **Scale:** how many users were affected? (2)

Root cause: why did it only appear in production (traffic pattern, data drift, edge-case input)? (3) **Detection**

and fix: how you found it (logs, traces, user reports) and what you changed. (4) **Prevention:** what monitoring, alerting, or regression tests you added so it gets caught before production next time. The fourth part is what separates mid-level from senior answers.

Q7. How do you estimate the RAM needed to deploy a model?

Use a simple, defensible formula: **Memory is roughly parameters times bytes-per-parameter, plus overhead.**

FP32 = 4 bytes per parameter, FP16/BF16 = 2, INT8 = 1, 4-bit quantized = 0.5. So a 7B model needs about 28 GB in FP32, 14 GB in FP16, 7 GB in INT8, and 3.5 to 4 GB in 4-bit.

Then add the **KV cache** (it grows with context length times batch size and can rival the weights for long contexts), activations, and framework overhead. Budget roughly 20 to 30% extra.

For disk storage, the weights file is parameters times bytes-per-parameter (a 7B FP16 checkpoint is about 14 GB).

Rule of thumb to quote: "For inference, plan about 2 GB of VRAM per billion parameters at FP16, plus 20 to 30% headroom for KV cache and overhead."

Pro tip: Mentioning KV cache here is a strong senior signal. Most candidates forget it.

@aiml_rohandevaki

Section 2: Fresher Fundamentals and GenAI Core

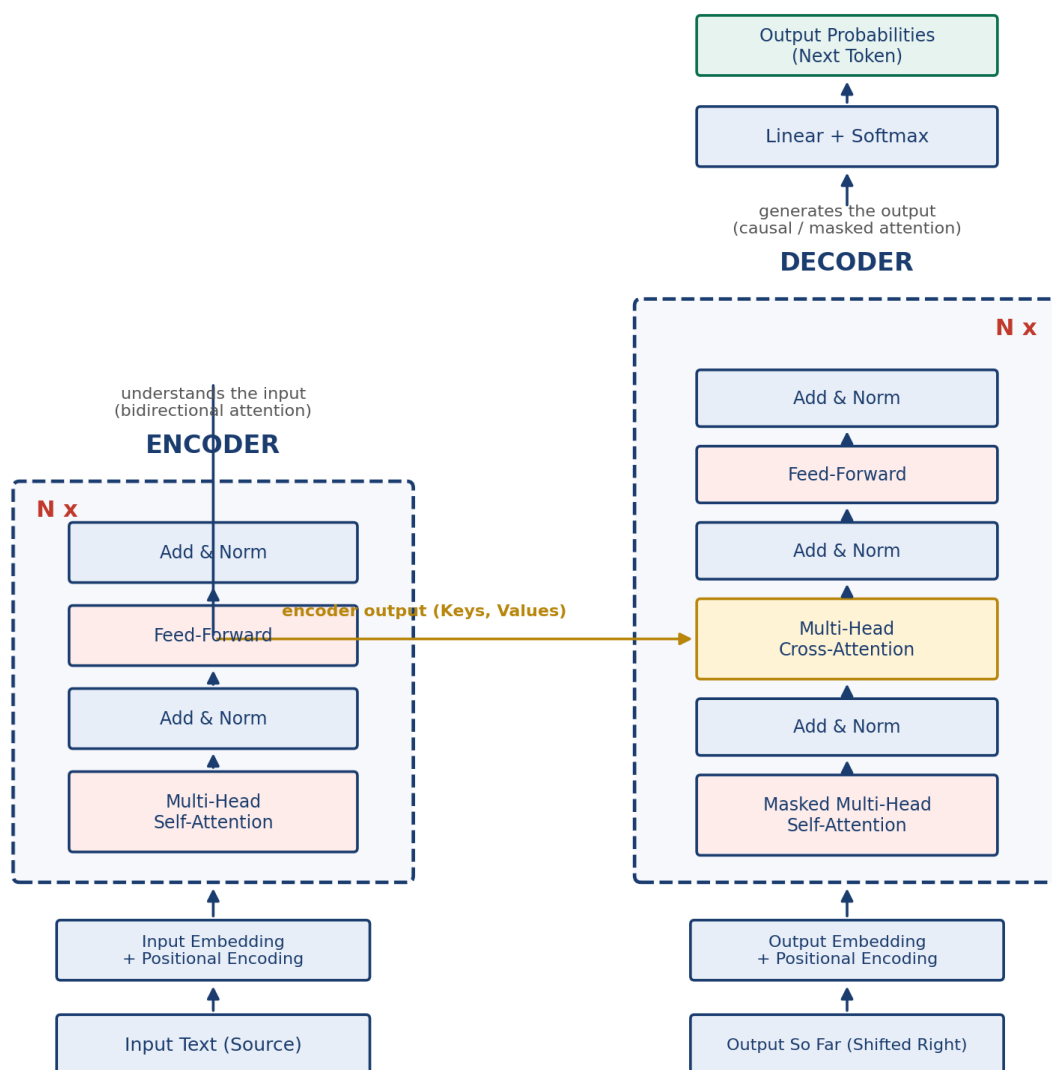
Q8. Explain the Transformer architecture.

Introduced in the paper “Attention Is All You Need” (2017). The original Transformer has two stacks: an **encoder** that understands the input, and a **decoder** that generates the output.

Shared building blocks: (1) **Tokenization and embeddings:** text becomes vectors. (2) **Positional encoding:** injects word order, since attention itself is order-agnostic. (3) **Multi-head self-attention:** every token computes Query-Key dot products against other tokens and takes a weighted sum of Values; multiple heads learn different relationship types in parallel. (4) **Feed-forward layers** after attention. (5) **Residual connections and layer normalization** for stable training.

What makes the two sides different: the encoder uses **bidirectional** self-attention (every token sees the whole input). The decoder uses **masked (causal)** self-attention (each token sees only previous tokens) plus a **cross-attention** layer, where the decoder's Queries attend over the encoder's output Keys and Values. That cross-attention link is how the generated output stays grounded in the input.

Transformer Architecture: Encoder + Decoder (as in “Attention Is All You Need”)



Data flows bottom to top in both stacks; each dashed block repeats N times, and cross-attention connects them.

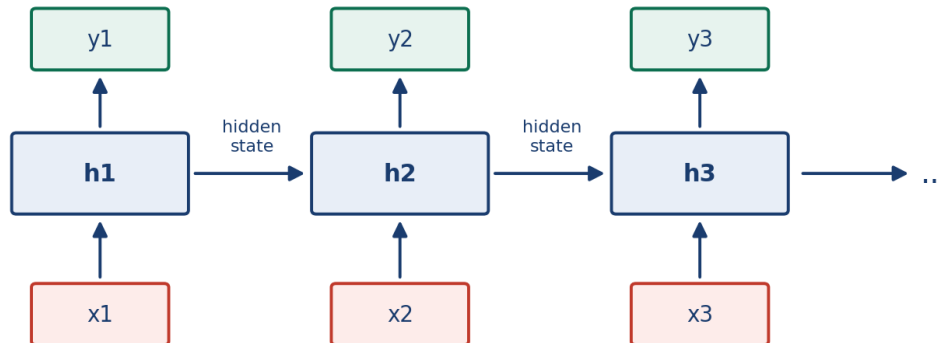
Close with the modern context: **BERT-style** models keep only the encoder (understanding tasks), **GPT-style** LLMs keep only the decoder with causal masking (generation), and **T5-style** models use the full encoder-decoder. Most production LLMs today are decoder-only.

@aiml_rohandevaki

Q9. Why are Transformers preferred over LSTMs and RNNs?

(1) Parallelism: RNNs process tokens sequentially, one step at a time. Transformers process the whole sequence at once, making training massively faster on GPUs.

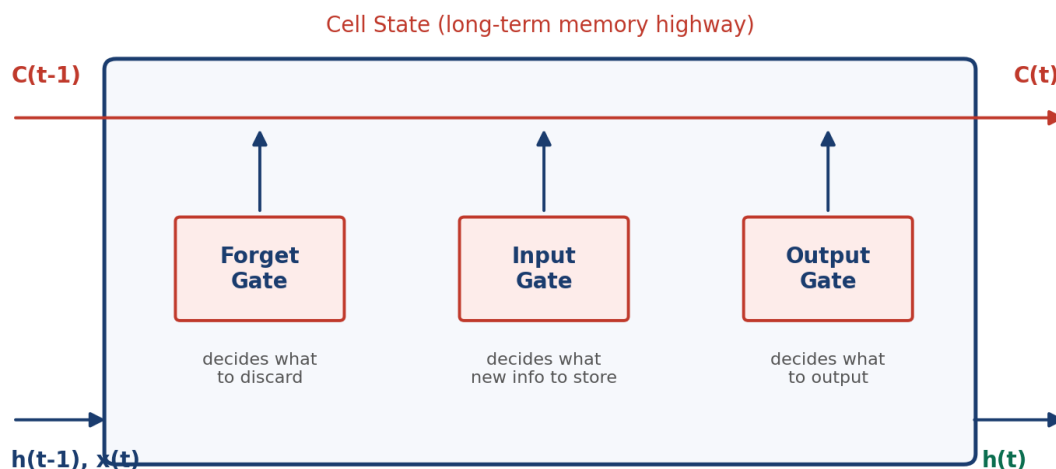
RNN (Unrolled): Tokens Processed One at a Time, Sequentially



RNN: the hidden state must pass through every step, so nothing can be parallelized.

(2) Long-range dependencies: attention connects any two tokens directly. RNNs suffer vanishing gradients over long sequences. LSTMs improve on this with a gated cell state, but still process sequentially and still degrade on very long contexts.

LSTM Cell: Gates Control What to Forget, Store and Output



LSTM: gates control the cell state to keep long-term memory, a fix for RNN forgetting, but still sequential.

(3) Scalability: Transformers scale predictably with data and parameters (scaling laws), which is what made LLMs possible. Trade-off worth mentioning: attention cost grows quadratically with sequence length, which is why long-context handling is still an active research area.

Q10. What is tokenization and why does it matter?

Tokenization splits text into smaller units called tokens: words, subwords, or characters. LLMs cannot read raw text; they process token IDs. Subword methods like BPE handle rare words and multiple languages while keeping vocabulary size manageable.

It also matters commercially: API pricing, context limits, and latency are all measured in tokens.

Q11. What are embeddings and how are they used in RAG?

Embeddings are dense numeric vectors that capture the meaning of text, so semantically similar texts land close together in vector space. In RAG, documents are chunked, embedded, and stored in a vector database. The user query is embedded the same way, and nearest-neighbor search retrieves the most relevant chunks to ground the answer.

Q12. What is a context window and what are its practical limits?

The context window is the maximum number of tokens the model can process in one pass. Bigger windows help, but attention cost grows quadratically with length, and models suffer from the “lost in the middle” problem where facts buried in the center of a long context get missed. This is why good retrieval beats stuffing everything into the prompt.

Q13. What does temperature do?

Temperature scales the logits before softmax. Low temperature (0 to 0.3) makes output focused and deterministic, good for extraction and code. Higher temperature (0.7 and above) increases randomness and creativity. Mention `top_p` as the related sampling knob to show depth.

Q14. What is the difference between a base model and an instruction-tuned model?

A base model is trained only on next-token prediction over large text corpora. It completes text but does not follow instructions well. An instruction-tuned model is further trained on instruction-response pairs, and often aligned with RLHF or DPO, so it behaves like an assistant. Almost everything you use in production is instruction-tuned.

Q15. What is hallucination and how do you reduce it?

Hallucination is when the model confidently generates false or fabricated content. It happens because the model predicts the most probable next token rather than verifying facts.

Reduction levers: ground answers with RAG and citations, instruct the model to say “I don't know,” lower the temperature for factual tasks, and add a verification step for high-risk outputs.

Q16. Zero-shot vs few-shot vs chain-of-thought prompting?

Zero-shot gives the task with no examples. Few-shot includes worked examples to lock in format and style. Chain-of-thought asks the model to reason step by step before answering, which improves accuracy on math, logic, and multi-step tasks.

Q17. Fine-tuning vs RAG vs prompt engineering: when to use which?

Prompt engineering when the model already knows enough and just needs steering. RAG when the problem is knowledge: private, frequently changing, or citable data. Fine-tuning when the problem is behavior: style, format, or domain tone. Production systems usually combine all three.

Q18. Encoder vs decoder transformers?

Encoders (like BERT) use bidirectional attention and are best for understanding tasks such as classification and similarity. Decoders (like GPT) use causal masking and generate left to right, best for generation. Most modern LLMs are decoder-only because generation is the main use case.

Q19. What is tool or function calling in LLMs?

The model does not just generate text; it can output a structured call to an external function (search, database query, API) with arguments. Your code executes the call and returns the result to the model. This is the foundation of every agent.

Pro tip: "How does an agent actually take actions?" is a favorite follow-up. This answer covers it.

Q20. What is an AI agent, and how is it different from a workflow?

A workflow is a predefined sequence of LLM calls and tools where the developer decides the path. An agent is a system where the LLM itself decides which tools to call, in what order, and when to stop, typically in a loop of reason, act, observe. Interviewers love this question because most candidates use the word "agent" loosely.

Q21. What chunking strategies do you know for RAG?

Fixed-size chunking with overlap is the baseline. Better options: sentence or paragraph-aware splitting, recursive character splitting, and semantic chunking that splits where meaning shifts.

Chunk size is a trade-off: too small loses context, too large dilutes retrieval precision and wastes tokens.

Pro tip: Saying you tuned chunk size against retrieval metrics is a strong hands-on signal.

Q22. How does similarity search work in a vector database?

The query embedding is compared against stored chunk embeddings using a metric like cosine similarity or dot product. At scale, exact search is too slow, so vector databases use approximate nearest neighbor indexes (like HNSW) that trade a tiny bit of recall for huge speed gains.

Combine this with metadata or payload filtering when queries contain exact fields like names and IDs.

Q23. Why choose RAG, and when?

RAG grounds the model in external, up-to-date, or private knowledge at inference time instead of relying on frozen training data. Choose RAG when knowledge changes frequently, when data is proprietary, when you need source citations, or when fine-tuning is too costly.

Q24. Where does traditional (naive) RAG fall short?

(1) **Exact-match and keyword queries:** pure semantic search misses rare names, IDs, and codes. The fix is hybrid search (BM25 plus vectors) and metadata filtering. (2) **Multi-hop questions** that need reasoning across documents. (3) **Bad chunking** that splits context mid-thought. (4) **Lost in the middle:** models under-weight content buried in long contexts. (5) No understanding of **relationships** between entities. (6) Retrieval quality caps generation quality: garbage in, garbage out.

Q25. GraphRAG vs standard RAG?

Standard RAG retrieves independent text chunks by vector similarity. GraphRAG first builds a knowledge graph of entities and relationships from the corpus, so retrieval can traverse relationships. It is ideal for questions like "how is X connected to Y?" or corpus-wide summarization that chunk retrieval cannot answer. The cost: building and maintaining the graph is expensive, so use it only when relationship-heavy queries justify it.

Q26. Explain the different types of RAG and their drawbacks.

Naive RAG: embed, retrieve, generate. Simple, but suffers from all the failures in Q24. **Advanced RAG:** adds query rewriting, hybrid search, and rerankers. Better accuracy, but more latency and cost per query. **Modular or Agentic RAG:** an agent decides when and what to retrieve, and can iterate and self-correct. Most capable, but slowest, costliest, and hardest to evaluate. **GraphRAG:** relationship-aware (see Q25), but expensive to build. Pick based on query complexity, latency budget, and cost.

Q27. What fine-tuning techniques do you know?

Full fine-tuning: update all weights. Best quality, enormous compute, and risk of catastrophic forgetting. **LoRA:** freeze base weights and train small low-rank adapter matrices, a tiny fraction of the trainable parameters. **QLoRA:** LoRA on top of a 4-bit quantized base model, which makes fine-tuning 7B to 70B models feasible on a single consumer GPU. **PEFT** is the umbrella term (LoRA, prefix tuning, adapters). Also know **instruction tuning** (instruction-response pairs) and **RLHF or DPO** (preference alignment) at a conceptual level.

Pro tip: If you have not fine-tuned in a real project, say so honestly and explain the concepts. Never bluff hands-on experience.

Section 3: Voice AI Agent Questions

Q28. How did you build a voice AI agent?

Describe the pipeline: **VAD** (voice activity detection), then **STT**, then **LLM** (agent logic, tools, memory), then **TTS**, with **streaming at every stage**. STT streams partial transcripts, the LLM streams tokens, and TTS starts speaking on the first sentence.

Also mention interruption handling (barge-in) and turn-taking. These details prove real hands-on experience.

Q29. Did you build custom components (STT, LLM, TTS) or use an integrated pipeline, and why?

Frame it as a trade-off. **Custom components**: full control, swap best-in-class models per stage, and tune each stage's latency, but you own the orchestration, streaming glue, and failure handling. **Integrated pipeline or speech-to-speech platform**: much faster to ship and latency comes pre-optimized, but less control and vendor lock-in.

State your choice and tie it to requirements. Example: "We needed a domain-tuned STT vocabulary, so custom won."

Q30. How do you improve word accuracy (reduce WER)?

Use the correct metric name: **Word Error Rate**. Levers: benchmark STT models on your own domain audio; add **custom vocabulary or keyword boosting** for domain terms and names; improve audio quality (noise suppression, echo cancellation, correct sample rate); use domain-adapted or fine-tuned STT where supported; and post-process with the LLM to correct obvious transcription errors in context.

Q31. How do you reduce TTFB / TTFT?

(1) **Stream everything**. Never wait for full outputs. (2) **Cut prompt size**. Smaller prompts mean faster prefill. My 30k to 10k token reduction was the single biggest win. (3) **Prompt caching** for the static system prompt. (4) **Model tiering**: use a smaller, faster model for simple turns and route hard turns to a bigger one. (5) Co-locate services to cut network hops. (6) Natural fillers like "Sure, let me check that" generated while the real answer computes.

Final advice: Interviewers build the entire round from your answers. Every claim in your resume and every number in your stories will get a "how exactly?" follow-up, so only include what you can defend three levels deep. Depth on a few real projects beats breadth on twenty buzzwords.

Coming in Part 2 (Advanced): System Design rounds, LLM evaluation and LLM-as-judge failure modes, cost optimization, guardrails and prompt injection, AI coding tools workflow, and behavioral rounds.

Found this useful? Follow [@aiml_rohandevaki](#) on Instagram for more AI/ML interview prep.